

Cahier des charges — Findy

1. Présentation du projet

Nom du produit : Findy

Type : Application web et mobile inspirée d'une expérience type TripAdvisor, centrée sur la découverte de lieux, avis utilisateurs et recommandations personnalisées.

Objectif : Permettre à un utilisateur de trouver facilement des restaurants, hôtels, activités et lieux d'intérêt, consulter des avis fiables, enregistrer ses favoris et planifier ses sorties ou voyages.

2. Vision produit

Findy doit être une plateforme simple, moderne et rapide qui aide les utilisateurs à : - découvrir des lieux proches ou dans une destination précise ; - lire et publier des avis ; - voir des notes, photos et commentaires ; - comparer plusieurs lieux ; - sauvegarder des endroits dans des listes ; - recevoir des recommandations adaptées à leurs goûts.

3. Cibles utilisateurs

Utilisateurs principaux

- Voyageurs
- Locaux cherchant des bonnes adresses
- Couples / groupes d'amis organisant des sorties
- Utilisateurs qui aiment noter et recommander des lieux

Utilisateurs secondaires

- Propriétaires d'établissements
 - Administrateurs / modérateurs de la plateforme
-

4. Problème à résoudre

Aujourd'hui, les utilisateurs doivent souvent jongler entre plusieurs apps pour : - chercher un lieu ; - lire des avis ; - comparer les options ; - enregistrer leurs trouvailles ; - organiser leur sortie.

Findy centralise tout cela dans une seule application avec une expérience claire et communautaire.

5. Objectifs du MVP

Le MVP doit permettre de valider : 1. l'intérêt des utilisateurs pour la découverte de lieux via avis ; 2. l'engagement autour des notes, commentaires et favoris ; 3. la pertinence d'un système de recherche + filtres + recommandations.

KPI MVP

- nombre d'inscriptions ;
 - taux d'utilisation de la recherche ;
 - nombre d'avis publiés ;
 - nombre de lieux sauvegardés ;
 - rétention à 7 jours et 30 jours.
-

6. Fonctionnalités principales

6.1 Authentification

- Inscription par email/mot de passe
- Connexion
- Connexion via Google / Apple (option MVP+)
- Réinitialisation de mot de passe
- Gestion du profil utilisateur

6.2 Recherche de lieux

- Barre de recherche par nom, ville, catégorie
- Résultats avec image, note, distance, type de lieu
- Recherche géolocalisée
- Suggestions automatiques

6.3 Filtres

- Catégorie (restaurant, hôtel, activité, café, musée...)
- Prix
- Note minimale
- Distance
- Horaires / ouvert maintenant
- Type de cuisine / options spécifiques

6.4 Fiche lieu

- Nom
- Photos
- Description
- Adresse + carte

- Horaires
- Note moyenne
- Avis utilisateurs
- Services / équipements
- Bouton favoris
- Bouton partage

6.5 Avis et notes

- Ajouter une note sur 5
- Rédiger un commentaire
- Ajouter des photos
- Modifier / supprimer son avis
- Signaler un avis

6.6 Favoris et listes

- Sauvegarder un lieu
- Créer des listes personnalisées
- Ajouter / retirer un lieu d'une liste
- Partager une liste (version future)

6.7 Recommandations

- Recommandations selon recherches passées
- Recommandations selon favoris
- Lieux similaires
- Tendances locales

6.8 Carte interactive

- Affichage des lieux sur carte
- Cliquer sur un pin pour voir un résumé
- Bascule liste / carte

6.9 Back-office admin

- Gérer les utilisateurs
- Gérer les lieux
- Modérer les avis
- Gérer les signalements
- Voir des statistiques globales

7. Fonctionnalités hors MVP

- Réservation intégrée
- Messagerie entre utilisateurs

- Suivi d'amis / réseau social
 - Gamification (badges, niveaux)
 - IA de recommandation avancée
 - Assistant de voyage / itinéraire
 - Version business pour établissements
-

8. Parcours utilisateur principaux

Parcours 1 — Découvrir un lieu

1. L'utilisateur ouvre l'app
2. Il saisit une ville ou active sa position
3. Il filtre les résultats
4. Il ouvre une fiche lieu
5. Il consulte avis, photos, carte
6. Il sauvegarde le lieu ou s'y rend

Parcours 2 — Laisser un avis

1. L'utilisateur se connecte
2. Il cherche un lieu visité
3. Il clique sur "Donner un avis"
4. Il met une note, un commentaire, des photos
5. Il publie

Parcours 3 — Créer une liste

1. L'utilisateur consulte un lieu
 2. Il clique sur "Enregistrer"
 3. Il choisit une liste existante ou en crée une nouvelle
 4. Le lieu est ajouté à sa collection
-

9. Rôles et permissions

Visiteur

- rechercher des lieux
- consulter les fiches
- lire les avis

Utilisateur connecté

- publier des avis
- noter des lieux
- enregistrer des favoris

- gérer son profil

Admin

- gérer contenu et modération
 - supprimer des avis ou comptes
 - valider / corriger des lieux
-

10. Exigences techniques

Frontend

- **Web** : Next.js
- **Mobile** : React Native / Expo
- **UI** : Tailwind CSS + bibliothèque de composants
- **Carte** : Mapbox ou Google Maps

Backend

- **API** : Node.js avec NestJS ou Express
- **Base de données** : PostgreSQL
- **ORM** : Prisma
- **Auth** : JWT + OAuth
- **Stockage images** : Cloudinary / S3

Admin

- Dashboard web sécurisé

Infra

- Hébergement frontend : Vercel
 - Hébergement backend : Railway / Render / AWS
 - Base de données : Supabase / Neon / RDS
-

11. Architecture fonctionnelle

Entités principales

- User
- Place
- Review
- Rating
- Favorite
- List

- Category
- Photo
- Report

Relations principales

- Un user peut écrire plusieurs reviews
 - Un place possède plusieurs reviews et photos
 - Un user peut avoir plusieurs listes
 - Une liste contient plusieurs lieux
-

12. Modèle de données simplifié

User

- id
- name
- email
- password_hash
- avatar_url
- created_at

Place

- id
- name
- description
- address
- city
- country
- latitude
- longitude
- category_id
- price_level
- average_rating
- created_at

Review

- id
- user_id
- place_id
- rating
- comment
- created_at
- updated_at

Favorite

- id
- user_id
- place_id

List

- id
 - user_id
 - title
 - description
 - created_at
-

13. Pages principales

- Accueil
 - Recherche / résultats
 - Fiche lieu
 - Connexion / inscription
 - Profil utilisateur
 - Mes favoris
 - Mes listes
 - Ajouter un avis
 - Admin dashboard
-

14. Exigences UX/UI

- Interface simple et visuelle
 - Design mobile-first
 - Recherche accessible dès l'accueil
 - Fiches lieu riches en images
 - Chargement rapide
 - Navigation fluide entre liste et carte
 - Système d'avis lisible et rassurant
-

15. Exigences non fonctionnelles

- Temps de réponse rapide
- Sécurité des données utilisateur
- Protection contre spam et faux avis
- Scalabilité pour ajout de villes et pays
- Responsive web + mobile

- Accessibilité de base (contraste, navigation claire)
-

16. Risques produit

- difficulté à obtenir des données de lieux au lancement ;
 - faible volume d'avis au début ;
 - modération des faux avis ;
 - dépendance aux APIs cartographiques / géolocalisation.
-

17. Stratégie de lancement

Phase 1

- Lancer sur une seule ville ou région
- Peu de catégories au départ
- Onboarding simple
- Collecte des premiers avis manuellement + communauté initiale

Phase 2

- Ajouter recommandations
 - Ajouter listes partagées
 - Étendre la couverture géographique
-

18. Roadmap technique proposée

Sprint 1 — Cadrage

- définir branding et UX
- définir schéma BDD
- maquettes principales
- setup repo frontend / backend

Sprint 2 — Base produit

- auth
- structure DB
- CRUD lieux
- recherche simple
- pages principales

Sprint 3 — Avis et favoris

- notes / commentaires

- favoris
- profils utilisateurs

Sprint 4 — Carte et filtres

- intégration carte
- filtres avancés
- géolocalisation

Sprint 5 — Admin

- modération
- gestion lieux / utilisateurs
- signalements

Sprint 6 — Stabilisation

- tests
- optimisation perf
- analytics
- déploiement MVP

19. Stack recommandée pour aller vite

Si vous êtes 2 devs, la stack la plus efficace serait : - **Frontend web** : Next.js - **Backend / DB / Auth** : Supabase - **ORM / requêtes** : Prisma ou client Supabase - **UI** : Tailwind - **Mobile plus tard** : Expo React Native - **Maps** : Mapbox - **Images** : Cloudinary

Cette version permet de sortir un MVP rapidement.

20. Découpage concret du code

Dossier frontend

- /app
- /components
- /features/auth
- /features/places
- /features/reviews
- /features/favorites
- /lib
- /services/api

Dossier backend

- /modules/auth
 - /modules/users
 - /modules/places
 - /modules/reviews
 - /modules/favorites
 - /modules/lists
 - /modules/admin
-

21. Priorité de développement

Priorité 1

- Auth
- Recherche
- Fiche lieu
- Avis

Priorité 2

- Favoris
- Listes
- Carte

Priorité 3

- Recommandations
 - Admin avancé
 - Social / partage
-

22. Définition du succès MVP

Le MVP est réussi si un utilisateur peut : 1. créer un compte ; 2. chercher un lieu ; 3. filtrer les résultats ; 4. consulter une fiche détaillée ; 5. enregistrer un lieu ; 6. publier un avis sans friction.

23. Suite recommandée

Après validation du cahier des charges, les prochaines étapes sont : 1. faire les user stories ; 2. dessiner les écrans ; 3. modéliser la base de données ; 4. définir les endpoints API ; 5. lancer le plan de sprints.